LIGHT-WEIGHT PROCESSES
Dissecting Linux Threads

This article, aimed at Linux developers and students of computer science, explores the fundamentals of threads and their implementation in Linux with Light-Weight Processes, aiding understanding with a code implementation.

Threads are the core element of a multi-tasking programming environment. By definition, a thread is an execution context in a process; hence, every process has at least one thread. Multi-threading implies the existence of multiple, concurrent (on multi-processor systems), and often synchronised execution contexts in a process.

Threads have their own identity (thread ID), and can function independently. They share the address space within the process, and reap the benefits of avoiding any IPC (Inter-Process Communication) channel (shared memory, pipes and so on) to communicate. Threads of a process can directly communicate with each other—for example, independent threads can access/update a global variable. This model eliminates the potential IPC overhead that the kernel would have had to incur. As threads are in the same address space, a thread context switch is inexpensive and fast. A thread can be scheduled independently; hence, multi-threaded applications are well-suited to exploit parallelism in a multi-processor environment. Also, the creation and destruction of threads is quick. Unlike fork(), there is no new copy of the parent process, but it uses the same address space and shares resources, including file descriptors and signal handlers.

A multi-threaded application uses resources optimally, and is highly efficient. In such an application, threads are loaded with different categories of work, in such a manner that the system is optimally used. One thread may be reading a file from the disk, and another writing it to a socket. Both work in tandem, yet are independent. This improves system utilisation, and hence, throughput.

A few concerns

The most prominent concern with threads is synchronisation, especially if there is a shared resource, marked as a critical section. This is a piece of code that accesses a shared resource, and must not be concurrently accessed by more than one thread. Since each thread can execute independently, access to the shared resource is not moderated naturally but using synchronisation primitives including mutexes (mutual exclusion), semaphores, read/write locks